

Báo cáo Thực nghiệm: LeNet-5 trên Tập dữ liệu MNIST

Assignment 4 · Khảo sát Kiến trúc Kinh điển · Scratch NumPy · TensorFlow/Keras · PyTorch

Tài liệu tổng hợp kết quả thực nghiệm chuyên sâu cho notebook `06_mnist_lenet.ipynb`. Mô hình cơ sở trong `02_mnist.ipynb` với cấu hình CNN 2conv+fc **được bảo tồn nguyên vẹn** làm hệ quy chiếu đối sánh khách quan và nhất quán.

Toàn bộ các chỉ số định lượng được trích xuất trực tiếp từ các file kết quả thực thi cục bộ `results/06_mnist_lenet.json` và `results/02_mnist.json`.

1. Khái lược về Kiến trúc LeNet-5

1.1 Bối cảnh lịch sử và Ý nghĩa

Kiến trúc **LeNet-5** được Yann LeCun cùng các đồng sự công bố vào năm 1998 qua công trình mang tính bước ngoặt *Gradient-Based Learning Applied to Document Recognition*. Đây là một trong những hệ thống mạng nơ-ron tích chập đầu tiên được áp dụng thành công trên quy mô công nghiệp thương mại, nhận diện tự động chữ số viết tay trên hàng triệu tờ séc ngân hàng tại Hoa Kỳ.

Ý nghĩa học thuật cốt lõi đối với đợt thực nghiệm này: **LeNet-5 được thiết kế và tinh chỉnh chuyên biệt cho tác vụ phân loại chữ số viết tay**. Do đó, tập dữ liệu MNIST chính là miền bài toán tối ưu tự nhiên của kiến trúc này, mang lại nền tảng đánh giá thực chất và chuẩn xác nhất.

1.2 Cấu trúc mạng tổng quát

Ảnh đầu vào $1 \times 28 \times 28$			
	↓		
C1	Conv 6 @ 5×5 , pad=2	→ $6 \times 28 \times 28$	Bản đồ đặc trưng không gian
	↓ ReLU		
S2	MaxPool 2×2	→ $6 \times 14 \times 14$	Lớp lấy mẫu con (subsampling)
	↓		
C3	Conv 16 @ 5×5 , pad=0	→ $16 \times 10 \times 10$	Bản đồ đặc trưng bậc hai
	↓ ReLU		
S4	MaxPool 2×2	→ $16 \times 5 \times 5$	Lớp lấy mẫu con (subsampling)
	↓ Flatten	→ 400 chiều	
F5	Dense 400 → 120	↓ ReLU	
F6	Dense 120 → 84	↓ ReLU	
out	Dense 84 → 10	(Điểm số Logits ngõ ra)	

Quy chuẩn ký hiệu trung thành với cấu trúc văn bản gốc năm 1998: tiền tố **C** biểu thị Convolution, **S** đại diện cho Subsampling (Pooling), và **F** đại diện cho Fully-connected.

1.3 Giải pháp kỹ thuật `pad=2` tại tầng C1

Trong thiết kế sơ khởi năm 1998, LeNet-5 tiếp nhận ảnh đầu vào chuẩn cỡ **32×32** rồi thực hiện tích chập không đệm (*valid*) với bộ lọc **5 × 5** để hạ xuống feature map **28 × 28**. Tuy vậy, ảnh số chuẩn trong MNIST có độ phân giải **28 × 28**. Hai phương án thích ứng kỹ thuật khả thi:

Giải pháp	Cơ chế xử lý	Kích thước ngõ ra
Đệm viền thủ công	Thêm dải 0 viền ngoài ảnh đưa 28 × 28 → 32 × 32 , sau đó áp dụng conv valid	28 × 28
Đệm tích chập trực tiếp (Phương án lựa chọn)	Cấu hình tham số <code>pad=2</code> ngay tại tầng Conv 5 × 5 trên kích thước ảnh gốc 28 × 28	28 × 28

Cả hai cách tiếp cận tạo ra các tensor ngõ ra hoàn toàn đồng nhất. Việc lựa chọn phương án thứ hai giúp tối ưu hóa đường ống dẫn dữ liệu và bảo đảm sự đồng bộ cấu trúc với notebook baseline.

1.4 Giải phẫu cấu trúc tham số

Tầng mạng	Số lượng tham số	Tỷ trọng trên toàn mạng
C1: Conv 1→6, kernel 5×5	156	0.25%
C3: Conv 6→16, kernel 5×5	2,416	3.92%
F5: Linear 400→120	48,120	77.98%
F6: Linear 120→84	10,164	16.47%
Output: Linear 84→10	850	1.38%
Tổng cộng	61,706	100.0%

Khối trích xuất đặc trưng không gian (C1 + C3) chỉ chiếm đúng 4.17% dung lượng tham số (2,572 trên 61,706). Hơn 94% trọng số mô hình tập trung ở khối phân loại Fully Connected, với riêng tầng F5 chiếm tới gần 78%. Đây là dấu ấn hình thái kinh điển của các mạng tích chập thời kỳ đầu: tầng tích chập đảm trách hầu hết các phép toán nhân-cộng ma trận nhờ cơ chế chia sẻ trọng số, trong khi các tầng kết nối dày đặc lại nắm giữ đại đa số dung lượng bộ nhớ tham số.

2. Khái niệm "Hiện đại hoá" (Modernized)

Kiến trúc triển khai trong khuôn khổ bài tập là **bản LeNet-5 đã qua hiện đại hóa kỹ thuật**, không giữ nguyên vẹn 100% nguyên mẫu 1998:

Thành phần	LeNet-5 nguyên bản (1998)	Phiên bản Hiện đại hoá (Thực nghiệm)
Hàm kích hoạt	<code>tanh</code>	ReLU
Thao tác Pooling	Average Pooling	Max Pooling
Cấu trúc Topo	6→16 feature maps, kernel 5×5, head 120→84→10	Bảo toàn chuẩn xác
Bộ tối ưu hóa	SGD cơ bản	Adam (chuẩn hoá chung với baseline)

Nguyên nhân kỹ thuật: Module tự xây dựng `scratch_nn.py` được thiết kế tối ưu với hai thành phần chuẩn là `ReLU` và `MaxPool2D`. Việc hiện đại hóa bảo đảm tính tương đồng thuật toán trên toàn bộ 3 framework (Scratch, Keras, PyTorch) mà không làm suy giảm đặc tính cốt lõi của mạng.

3. Thiết lập Thử nghiệm và Đảm bảo Tính Công bằng

Quy chuẩn so sánh tuân thủ nghiêm ngặt nguyên lý công bằng kiểm thử từ bài giảng:

Cùng Dataset + Cùng Phân chia Train/Test + Cùng Kiến trúc + Siêu tham số tương đương.

Thông số thiết lập	Giá trị thực nghiệm
Tập con đối sánh 3 framework	10,000 mẫu train / 2,000 mẫu test
Tập chuẩn toàn phần tham chiếu	60,000 mẫu train / 10,000 mẫu test
Số lượng Epoch	5 epochs
Kích thước Batch	64
Thuật toán & Tốc độ học	Adam, Learning Rate = 1e-3
Random Seed	42 (tái lập hạt giống ngẫu nhiên đồng bộ)
Chuẩn hoá dữ liệu	Tính thống kê μ, σ riêng trên tập train

3.1 Quy chuẩn đánh giá thời gian thực thi

Lưu ý bản chất phần cứng: **PyTorch tận dụng nhân CUDA trên GPU rời NVIDIA GeForce RTX 3060, trong khi TensorFlow vận hành trên CPU** (do các bản phân phối TensorFlow native gần đây trên Windows không kèm GPU runtime). Chính vì vậy, chỉ số thời gian huấn luyện `train_seconds` phản ánh hiệu năng thiết bị tại chỗ, không dùng làm thước đo đánh giá trực diện giữa hai thư viện cấp cao.

3.2 Đối chiếu Triển khai trên Ba Khung làm việc

Thành phần	Scratch (NumPy)	TensorFlow/Keras	PyTorch
Khởi tạo mô hình	<code>S.Sequential([...])</code>	<code>keras.Sequential([...])</code>	<code>nn.Module</code> kế thừa
Tích chập C1 / C3	<code>S.Conv2D(k=5)</code> via <code>im2col</code>	<code>layers.Conv2D(5)</code>	<code>nn.Conv2d(5)</code>
Lấy mẫu S2 / S4	<code>S.MaxPool2D</code> định tuyến <code>argmax</code>	<code>layers.MaxPooling2D</code>	<code>nn.MaxPool2d</code>
Tầng kết nối Dense	<code>S.Dense</code> ma trận tường minh	<code>layers.Dense</code>	<code>nn.Linear</code>
Đạo hàm ngược	Đạo hàm giải tích viết tay + <code>col2im</code>	Tự động qua <code>GradientTape</code>	Tự động qua <code>autograd</code>
Vòng lặp huấn luyện	For-loop tường minh	<code>model.fit()</code> đóng gói	For-loop tường minh
Nền tảng tính toán	CPU (NumPy BLAS)	CPU	GPU (RTX 3060 CUDA)

4. Kết quả Thực nghiệm Định lượng

4.1 So sánh Ba Nền tảng trên Tập con 10k

Nền tảng thực thi	Số tham số	Accuracy	Macro-F1	Thời gian (s)	Nền tảng phần cứng
PyTorch	61,706	97.85%	0.9784	2.60	GPU (RTX 3060)
TensorFlow/Keras	61,706	97.65%	0.9763	4.13	CPU
Scratch (NumPy)	61,706	97.50%	0.9748	728.21	CPU

Cả ba framework kiểm chứng độc lập đều ghi nhận **chính xác 61,706 tham số**, khớp tuyệt đối với tính toán giải tích lý thuyết. Biên độ chênh lệch độ chính xác giữa ba nền tảng chỉ là **0.35 điểm phần trăm** (từ 97.50% đến 97.85%), khẳng định tính nhất quán toán học hoàn hảo của giải thuật bất chấp sự phân hóa về tầng trừu tượng phần mềm.

4.2 Kiểm chứng trên Toàn bộ Tập dữ liệu (60k mẫu)

Nền tảng thực thi	Accuracy	Macro-F1	Thời gian huấn luyện (s)
TensorFlow/Keras	98.70%	0.9869	31.67
PyTorch	98.68%	0.9867	17.51

Khi quy mô tập dữ liệu mở rộng từ 10k lên đủ 60k ảnh, độ chính xác tăng thêm xấp xỉ **+1.0 đến +1.2 điểm phần trăm**, đạt ngưỡng ~98.70% ở cả hai framework chính thống.

5. Đối đầu Thực nghiệm: LeNet-5 vs CNN 2conv+fc Baseline

Mô hình cơ sở CNN 2conv+fc (Notebook 02) cấu thành từ 2 tầng conv 3×3 với 16 và 32 kênh dẫn vào Linear(1568 $\rightarrow$ 10), sở hữu tổng cộng **20,490 tham số**.

5.1 Bảng Đối sánh Chi tiết

Tập dữ liệu	Nền tảng	Baseline (20k params)	LeNet-5 (61k params)	Độ chênh lệch
MNIST (10k subset)	Scratch (NumPy)	98.10%	97.50%	-0.60%
MNIST (10k subset)	TensorFlow/Keras	97.40%	97.65%	+0.25%
MNIST (10k subset)	PyTorch	98.35%	97.85%	-0.50%
MNIST (Toàn phần 60k)	TensorFlow/Keras	98.65%	98.70%	+0.05%
MNIST (Toàn phần 60k)	PyTorch	98.70%	98.68%	-0.02%

Chênh lệch hiệu năng trung bình tổng thể: -0.16 điểm phần trăm (Hiệu quả thực tế tương đương).

5.2 Phân tích Chuyên sâu

- Gia tăng gấp ba dung lượng tham số không tạo ưu thế phân loại vượt trội:** LeNet-5 chứa 61,706 tham số (nhiều gấp 3.01 lần Baseline với 20,490 tham số), song độ chính xác thu về gần như tương đương. Hiện tượng này minh chứng rõ ràng: **Với một bài toán có độ phân tách hình học cao như MNIST, khả năng biểu diễn của mạng tích chập nhanh chóng tiệm cận ngưỡng bão hòa.**
- Dữ liệu lớn giúp hai kiến trúc hội tụ về một mặt bằng chung:** Trên tập 60k toàn diện, sai khác giữa hai mô hình chỉ là 0.02% - 0.05% (tương đương chênh lệch vốn vụn 2 đến 5 ảnh kiểm thử trên 10,000 ảnh), hoàn toàn nằm gọn trong khoảng biến thiên ngẫu nhiên của việc khởi tạo trọng số.

5.3 Nghịch lý Thú vị: Số tham số lớn hơn nhưng Tốc độ tính toán nhanh hơn

Mô hình	Tổng số tham số	Thời gian huấn luyện Scratch CPU (s)
CNN 2conv+fc (Baseline)	20,490	998.52
LeNet-5	61,706	728.21

Dù sở hữu số tham số lớn gấp 3 lần, **LeNet-5 lại hoàn tất quá trình huấn luyện nhanh hơn Baseline khoảng 27% trên cùng môi trường CPU NumPy.**

Bản chất hiện tượng bắt nguồn từ **sự phân định rạch ròi giữa số lượng tham số lưu trữ và độ phức tạp phép tính tích chập động (Conv FLOPs / MACs)**:

Baseline (CNN 2conv+fc):

C1: $28 \times 28 \times 16 \times 1 \times 3 \times 3 = 112,896$ MACs

C2: $14 \times 14 \times 32 \times 16 \times 3 \times 3 = 903,168$ MACs

Tổng tích chập $\approx 1,016,064$ MACs

LeNet-5:

C1: $28 \times 28 \times 6 \times 1 \times 5 \times 5 = 117,600$ MACs

C3: $10 \times 10 \times 16 \times 6 \times 5 \times 5 = 240,000$ MACs

Tổng tích chập $\approx 357,600$ MACs

⇒ LeNet-5 tiêu tốn ít hơn tới 2.84 lần số phép tính nhân-cộng tích chập!

Mỗi trọng số ở tầng Dense chỉ tham gia đúng một phép nhân cho mỗi mẫu dữ liệu đầu vào. Ngược lại, mỗi trọng số kernel trong tầng tích chập được tái sử dụng hàng trăm lần qua các vị trí không gian. Baseline duy trì số kênh dày hơn (16 rồi 32) nên gánh nặng tính toán của số trượt áp đặt lên CPU là rất lớn; trong khi LeNet-5 có số kênh tích chập tương đối mỏng (chỉ 6 và 16) nên giảm tải đáng kể cho phép toán `im2col` và `col2im`.

Kết luận cốt lõi: Số lượng tham số đo lường dung lượng lưu trữ tĩnh, chứ không đồng nhất với khối lượng tính toán động (FLOPs / Latency).

6. Đúc kết Toàn diện

1. **LeNet-5 khẳng định vị thế tối ưu trên tác vụ nhận dạng chữ số viết tay MNIST** (đạt 97.85% trên tập con 10k và 98.70% trên tập toàn phần 60k), minh chứng cho tính chuẩn mực trong thiết kế lịch sử của Yann LeCun.
2. **Ưu thế dung lượng tham số không tự động chuyển hóa thành ưu thế độ chính xác** khi mạng đã vượt ngưỡng bão hòa của bài toán.
3. **Bài học cấu trúc phần cứng - thuật toán**: Mật độ kênh tích chập chi phối thời gian chạy nhiều hơn kích thước các tầng fully-connected.
4. **Tính toàn vẹn thuật toán được bảo đảm**: Cả 3 nền tảng triển khai (bao gồm cả mã nguồn tự xây dựng từ đầu) đều đồng thuận về cấu trúc tham số và động học hội tụ.